

第 22 组第 3&4 周进度汇报

一、两周工作概述

第 3 周我们小组实现了纯云端 AI 架构的完整语音交互链路，对接小米官方 API (mimo-v2.5) 打造了“唤醒→采集→识别→解析→执行→反馈”的全流程语音控制能力，各项功能达到设计指标。

第 4 周我们小组完成了断网检测与功能降级逻辑，也就是“离线 + 云端”双模式语音交互架构和网络状态监控与离线容错机制，实现了灯光全功能语音控制、真实天气查询、日期时间播报等功能。

二、核心任务完成情况

第 3 周核心任务具体完成情况如下：

1. 小米 AI API 全链路对接与云端语音交互系统实现

我们开发了独立的中间件模块 (xiaomi_ai.py) 完成小米 AI 开放平台对接并构建了完整的云端语音交互引擎：

实现了小米 ASR 语音识别、NLP 自然语言理解、TTS 语音合成及通用技能调用接口的标准化封装，通过 xiaomi_config.json 统一管理 API 密钥、服务地址与模型参数；实现了“按键唤醒→5 秒同步录音→云端 ASR 转写→NLP 意图解析→设备指令执行→TTS 语音反馈”的全自动化流程，支持“打开风扇”、“关闭风扇”、“风扇调高速”、“播报温度”等 10 余种常用指令；支持开风扇 (fan_on)、关风扇 (fan_off)、调转速 (fan_speed)、报温度 (announce_temperature)、报湿度 (announce_humidity)、切换温/湿/光照显示 (switch_display) 等 8 类核心意图；实现了最多 10 条对话记录的环形缓冲区存储，可以自动记录用户指令、系统响应与系统状态并实时同步至 E5 触控屏显示。

2. 场景化智能联动算法重构与差异化控制

我们对原有智能联动逻辑进行了全面重构，实现了基于场景的精细化、差异化控制策略：

将场景模式优化为“工作 (0)、休闲 (1)、睡眠 (2)”三类对应不同的控制策略：

工作模式：启用自动风扇，根据温度分档调节 (26℃以下关、26~28℃ 40%、28~30℃ 60%、30℃以上 80%)。

休闲模式：默认维持 20% 低噪音的风扇档位且手动控制优先。

睡眠模式：默认关闭风扇，仅当温度超过 35℃时强制开启 60% 档。

在通过按键手动控制风扇后系统就会被标记为手动覆盖状态，自动模式暂时失效；在切换场景时系统自动清除手动覆盖标记并恢复场景自动控制；不同场景设置不同的无人休眠阈值（工作 3 分钟、休闲 5 分钟、睡眠 1 分钟）；工作模式下环境异常时自动触发语音告警，睡眠模式完全关闭仅通过 E1 RGB 灯颜色提示状态。

3. E5 触控屏交互系统升级与语音界面开发

我们对 E5 Web 交互系统进行了升级，新增了完整的语音交互界面与对话历史展示功能：

前端界面新增独立的语音助手控制模块包含“语音播报开关”、“播报环境状态”、“唤醒语音交互”三个核心按键；在 e5_status.json 中新增了最近识别文本（last_recognized_text）、最近解析意图（last_nlp_intent）、对话历史数组（voice_history）三个字段，实现了语音状态的全量同步。

第 4 周核心任务具体完成情况如下：

1. "离线 + 云端" 双模式语音交互系统升级

我们重构了语音交互核心架构，实现了离线关键词匹配优先、云端 NLP 补充的双模式处理机制以解决网络中断时语音功能完全失效的问题：

在 xiaomi_ai.py 中实现了覆盖所有核心功能的本地关键词匹配规则，包括灯光控制（开 / 关 / 亮度 / 颜色）、风扇控制（开 / 关 / 调速）、环境信息播报（温度 / 湿度 / 光照）、场景切换、日期时间查询等 12 类意图，在网络离线时仍能处理 90% 以上的常用指令；当云端 ASR 或 NLP 调用失败时，系统自动切换到离线模式，基于原始语音识别文本进行关键词匹配并给出明确的离线模式提示；完善了录音失败、识别失败、解析失败等异常场景的语音反馈；在 execute_voice_intent 函数中增加了 fallback_intent 机制，即使 NLP 未返回有效意图也能通过原始文本中的关键词推断用户意图并执行。

2. 真实天气查询与多维度信息服务扩展

我们真正实现了基于真实气象数据的天气查询服务并扩展了日期时间查询功能：

在 xiaomi_ai.py 中新增了 get_real_weather 函数，通过公共 API（wttr.in）获取全球城市的实时天气数据并支持温度、风向风力、湿度等信息的格式化输出；支持“天津天气”、“北京的天气”等带城市名的查询指令，未指定城市时自动使用 xiaomi_config.json 中配置的默认城市天津；在网络完全离线时可以给出明确的网络状态提示；新增 query_date 和 query_time 两个意图，支持“今天几号”、“现在几点”等常用查询，通过系统本地时间返回结果。

3. 网络状态监控与离线容错机制

我们新增了独立的网络监控子系统，实现了网络状态的实时检测与全系统状态同步：

在 main.cpp 中实现了 network_monitor_thread，每 30 秒检测一次网络连通性（通过连接 8.8.8.8:53 Google 公共 DNS 服务的地址和端口），更新全局网络状态标志；使用互斥锁保护网络状态变量的读写，避免多线程竞争问题；对 E5 Web 交互系统（index.html 和 refresh.js）进行了细节优化，网络状态写入 e5_status.json 和 e5_ui.txt 并实时同步到 E5 触控屏前端显示；在语音交互和 Web 界面中明确提示当前网络状态，离线时告知用户可用的功能范围。